

CONCEPTOS DE DISEÑO

El diseño es un proceso creativo de transformación de un problema en solución:

- se basa en los requerimientos de los clientes
- identifica componentes del software y relaciones
- debe ser modelado y documentado.
- Agrupa principios, conceptos y prácticas que llevan al desarrollo de un sistema o producto de calidad

PROCESO DE DISEÑO

Es un proceso iterativo que consiste en un alto nivel de abstracción, refinamiento y menos nivel de abstracción resultante. Para ello hay que comprender y definir el contexto y las interacciones, diseñar la arquitectura del sistema, identificar los principales objetos del sistema, desarrollar modelos de diseño y especificar interfaces.

El modelo de diseño incluye representaciones, arquitecturas e interfaces en el nivel de componente y despliegue. Se evalúa para identificar errores, inconsistencias u omisiones, o bien alternativas superadoras, y también para corroborar si la implementación del modelo está dentro del plazo y costo establecidos.

CONCEPTOS FUNDAMENTALES

ABSTRACCIÓN

La solución a cualquier problema se presenta en varios niveles de abstracción:

- Nivel alto: solución en términos del lenguaje del ambiente del problema
- Nivel bajo: descripción detallada de la solución (para implementación)

La abstracción de procedimientos es una secuencia de instrucciones que implica funciones sin detalle de procedimientos. La abstracción de datos es un conjunto de datos con nombre, que describe a un objeto de datos.

ARQUITECTURA

Es la estructura global del software y cómo proporciona integridad conceptual al sistema. Representa los componentes que tiene el software, como interactúan y la estructura de los datos que utilizan. Comprende patrones arquitectónicos y la reutilización de componentes

PATRONES

Un patrón de diseño describe una estructura de diseño que resuelve un problema de diseño particular, dentro de un contexto específico. Provee información que permite al diseñador determinar:

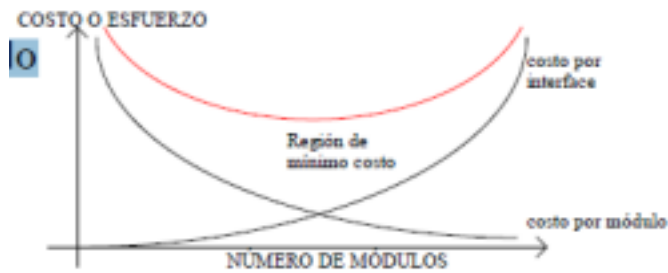
- si el patrón es aplicable
- si puede reutilizarse
- si se puede usar como guía para desarrollar algún patrón similar con estructura diferente.

DIVISIÓN DE PROBLEMAS

Cualquier problema complejo puede manejarse con más facilidad si se subdivide en elementos susceptibles de resolverse de manera independiente. Implica menor esfuerzo y menos tiempo

MODULARIDAD

El software se divide en componentes con nombres distintos y abordables por separado (módulos) que se integran para satisfacer los requerimientos del problema. Se facilita comprensión y reduce el costo para resolverlo



OCULTAMIENTO DE INFORMACIÓN

Los módulos se caracterizan por:

- decisiones de diseño que se ocultan una de otras.
- La información (algoritmos y datos) contenida en un módulo es inaccesible para los que no necesiten de ella.
- Ser independientes que intercambien solo la información necesaria para lograr la función del software, minimizando la propagación de errores

INDEPENDENCIA FUNCIONAL

Cada módulo:

- Resuelve un subconjunto específico de requerimientos
- Tiene una interfaz sencilla.
 - Reduce efectos secundarios por modificaciones.
 - Reduce la probabilidad de propagación del error.
 - Módulos reutilizables.

Criterios cualitativos:

- Cohesión: un módulo cohesivo ejecuta una sola tarea y requiere interactuar poco con otros componentes
- Acoplamiento: es la interconexión entre módulos de una estructura de software. Depende de:
 - La complejidad de la interfaz entre módulos
 - El grado en el que se hace referencia a un módulo
 - Qué datos pasan por la interfaz.

REFINAMIENTO

Se trabaja sobre el enunciado original (conceptual) dando más detalles conforme tiene lugar la elaboración sucesiva

REDISEÑO

Es el proceso de cambiar un sistema de software en forma tal que no se altera el comportamiento externo del código, pero si se mejora su estructura interna. Se examina el diseño existente para obtener un diseño mejor, analizando redundancias, elementos de diseño no utilizados, algoritmos ineficientes o innecesarios y estructuras de datos inapropiadas. Se pueden emplear métodos ágiles.

MODELO DE DISEÑO

Los elementos del modelo de diseño usan diagramas UML. Esos diagramas se refinan como parte del diseño, dándose más detalles específicos de la implementación y haciendo énfasis en la estructura y en el estilo arquitectónico, en los componentes que residen en la arquitectura y en las interfaces entre los componentes y el mundo exterior.

ELEMENTOS DEL DISEÑO DE DATOS

Se crea un modelo de datos que se representa en un nivel de abstracción equivalente al punto de vista del usuario. Este modelo se refina en forma progresiva hacia representaciones más específicas que puedan ser implementadas.

ELEMENTOS DEL DISEÑO ARQUITECTÓNICO

Indican la estructura, funcionamiento e interacción de las partes, brindando una visión general del software. (similares a los planos de un edificio o de construcción de una casa)

ELEMENTOS DE DISEÑO DE LA INTERFAZ

Proveen información hacia adentro y hacia afuera del sistema, y cómo están comunicados los diferentes componentes de la arquitectura. Incluye la interfaz de usuario, las interfaces externas que tienen que ver con otros sistemas y las interfaces internas que involucran a otros componentes del diseño.

ELEMENTOS DE DISEÑO EN EL NIVEL DE LOS COMPONENTES

Se describen los detalles internos de cada componente. Definen la estructura de datos para todos los objetos de datos locales y detalles algorítmicos para todo el procesamiento que tiene lugar en el componente, así como la interfaz. Son equivalentes a los planos detallados de cada habitación de una casa, con plomería y cableado de cada cuarto, detalles de pisos, etc.

ELEMENTOS DEL DISEÑO DEL DESPLIEGUE

Indican la forma en la que se acomodará la funcionalidad del software y los subsistemas en el ambiente físico.

DESCRIPCIÓN DEL DISEÑO DEL SOFTWARE (SDD)

El resultado del proceso de diseño es la documentación denominada “Descripción del diseño del software”. Un estándar empleado para desarrollar esta documentación en forma normalizada es el IEEE Std. 1016-1998.

IEEE STD. 1016-1998

Describe prácticas recomendadas para describir los diseños de software. Especifica la información que debe contener, y recomienda cómo organizarla. Puede emplearse en software comercial, científico o militar sin limitaciones por el tamaño, complejidad o nivel de criticidad. El estándar no establece ni limita determinadas metodologías de diseño, gestión de la configuración o aseguramiento de la calidad.

Ejemplo de una posible organización de la información en una SDD

1.- Introducción	4.- Descripción de las dependencias
1.1 Propósito	4.1 Dependencias intermódulares
1.2 Alcance	4.2 Dependencias inter-procesos
1.3 Definiciones y acrónimos	4.3 Dependencias de los datos
2.- Referencias	5.- Descripción de interfaces
3.- Descomposición de la información	5.1 Interfaces entre módulos
3.1 Descomposición modular	5.1.1 Interfaz del módulo 1
3.1.1. Descripción del módulo 1	5.1.2 Interfaz del módulo 2
3.1.2. Descripción del módulo 2	5.2 Interfaces entre procesos
3.2 Descomposición de los procesos	5.2.1 Interfaz del proceso 1
3.2.1. Descomposición del proceso 1	5.2.2 Interfaz del proceso 2
3.2.2. Descomposición del proceso 2	6.- Diseño detallado
3.3 Descomposición de los datos	6.1 Diseño detallado de los módulos
3.3.1. Descripción de la entidad 1	6.1.1 Detalle del módulo 1
3.3.2. Descripción de la entidad 2	6.1.2 Detalle del módulo 2
	6.2 Diseño detallado de los datos
	6.2.1 Detalle de la entidad 1
	6.2.2 Detalle de la entidad 2

DISEÑO ORIENTADO A OBJETOS

El paradigma orientado a objetos proporciona mejores herramientas para obtener un modelo del mundo real cercano a la perspectiva del usuario, interactuar fácilmente con un entorno de computación (empleando metáforas familiares) y facilitar la modificación y extensión de los componentes sin codificar de nuevo desde cero.

Existen diferentes modelos:

- Modelos estructurales: describen la estructura básica del sistema utilizando clases de objetos y sus relaciones, como el diagrama de clases.
- Modelos dinámicos: describen la estructura dinámica del sistema y muestran la interacción entre sus objetos. Diagrama de transición de estado y diagrama de interacción.

Un sistema orientado a objetos contiene objetos que interactúan, mantiene su propio estado local y proveen operaciones en ese estado. Los objetos están asociados con cosas y las representaciones de su estado son privadas y no pueden accederse desde afuera del objeto.

DIAGRAMA DE CLASES

Constituyen una expresión del modelo OO y sus elementos son clases, atributos, operaciones, relaciones, cardinalidad y roles. Cuando se construye un modelo lo primero que se mira es el mundo real. Se identifican los objetos esenciales y se representan como clases representándolas en cajas.

OBJETO

Un objeto es un objeto, abstracción o cosa que tiene un cierto significado para una aplicación. Un objeto es una instancia u ocurrencia de una clase. Los objetos y sus relaciones se describen mediante diagramas de instancias.

CLASE

Una clase es la descripción de un grupo de objetos que comparten atributos, operaciones relaciones y semántica. Poseen propiedades similares (atributos), comportamiento común (operaciones y diagrama de estado) y semántica común. Establece el mismo tipo de relaciones con otros componentes del modelo.

- Las clases proporcionan un mecanismo para compartir la estructura entre objetos similares. • Debe tener un nombre único.
- Un atributo expresa una propiedad particular de una clase
- Una operación refleja un servicio del objeto
- Los atributos y operaciones pueden ser mostrados de manera exhaustiva en los compartimentos de clases. • Pueden tener una contrapartida real o ser entidades conceptuales.
- Las clases y sus relaciones se describen mediante diagramas de clases.

ATRIBUTO

Un atributo es una propiedad de una clase a la que se le asigna un nombre y que contiene un valor para cada objeto de la clase.

OPERACIONES Y MÉTODOS

Una operación es una función o procedimiento que puede ser ejecutada por un objeto o aplicada a un objeto, que es su argumento implícito. A su nombre se le puede agregar detalles tales como la lista de argumentos o el tipo de resultado. Su implementación para una clase se denomina método.

IDENTIFICACIÓN DE LAS CLASES

Se identifican los objetos esenciales y luego se hace un refinamiento. Las descripciones de casos de uso colaboran en la identificación de objetos y operaciones en el sistema. Además de hacer un análisis gramatical de una descripción en lenguaje natural del sistema a ser construido: los objetos y atributos son sustantivos, mientras que las operaciones y servicios son verbos. Usar entidades del dominio aplicación

RELACIONES

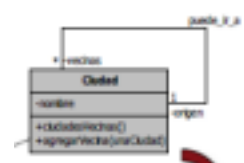
Es una conexión semántica entre clases del modelo, que tiene nombre, cardinalidad y roles. La identificación de relaciones permite reconocer la manera en que colaboran los distintos objetos en el dominio.

- Navegabilidad: es posible pasar de un objeto de la clase origen a uno varios objetos de la clase destino, dependiendo de la multiplicidad
- Multiplicidad: especifica la cantidad de objetos que pueden participar en una relación en cualquier momento.
- Rol: el rol de un objeto indica el papel que juega en su relación con otros objetos. Los nombres de los roles enriquecen la semántica de una relación. Son opcionales y aparecen como sustantivos, se escriben cerca de la clase a la cual califican.

RELACIÓN DE ASOCIACIÓN

Una asociación conecta dos clases y refleja que las mismas están relacionadas en el dominio de aplicación.

Una asociación es reflexiva cuando objetos de la clase tienen vínculos con objetos de la misma clase.



RELACIÓN DE AGREGACIÓN

Permite modelar una relación parte-todo en la cual un objeto es propietario de otro objeto pero no en exclusividad. Uno de los extremos cumple un rol predominante respecto del otro extremo (relación no simétrica)

No realiza ninguna afirmación sobre el ciclo de vida de las partes involucradas: el conjunto puede existir independientemente de las partes y las partes pueden existir

independientemente del conjunto. El conjunto está incompleto si falta alguna de las

partes. Es posible tener propiedad compartida de las partes por varios conjuntos

RELACIÓN DE COMPOSICIÓN

Permite modelar una relación parte-todo en la cual un objeto es propietario de otro

objeto en exclusividad. Uno de los extremos cumple un rol predominante respecto del otro extremo (relación no simétrica).

Las partes con multiplicidad variable pueden ser creadas luego del todo, pero una vez creadas no podrán persistir si el todo desaparece (se asocian los ciclos de vida del todo y la parte). Expresa una relación de pertenencia: las partes pueden solamente pertenecer a un conjunto. El conjunto tiene responsabilidad única para la disposición de todas sus partes (para su creación y destrucción): si se destruye el conjunto, debe destruir todas sus partes.

RELACIÓN DE GENERALIZACIÓN

Expresa clasificación entre un elemento más general y un elemento más específico. Es la relación entre una clase (superclase) y una o más variaciones de esta clase (subclases). Una instancia de una subclase es instancia de las superclases de esa clase. Los atributos, las operaciones, las relaciones y las restricciones definidas en la superclase se heredan íntegramente en la subclase.

La generalización es la relación estructural que permite la existencia del mecanismo de herencia. Las propiedades heredadas pueden reutilizarse o redefinirse en la subclase. La subclase puede definir nuevas relaciones no presentes en la superclase. El caso en que una subclase tiene múltiples superclases inmediatas se denomina herencia múltiple.

CLASE ASOCIACIÓN

Es una asociación cuyos enlaces pueden participar en asociaciones posteriores. Relación muchos a muchos entre

dos clases, donde existen atributos que no se pueden acomodar fácilmente en ninguna de las clases

Tiene características de asociación y de clase:

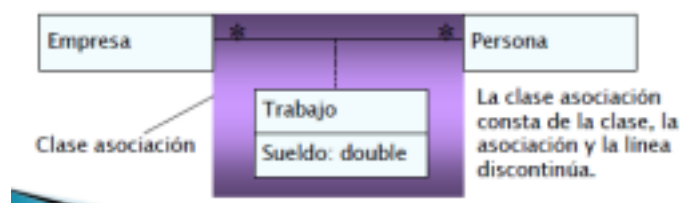
- Como el enlace de una asociación, las instancias de una clase de asociación obtienen su identidad de las instancias de las clases que constituyen.
- Como una clase, puede participar en asociaciones.

OTROS CONCEPTOS

RESPONSABILIDAD

Es una declaración general sobre un objeto de software:

- una acción que realiza el elemento
- un conocimiento que mantiene sobre algo (datos privados encapsulados, objetos relacionados y cosas que se



pueden derivar o calcular)

- una decisión importante que afecta a otro objeto de software.
- servicio que una clase ofrece
- contrato u obligación que tiene una clase con sus clientes (objetos que piden un servicio)

MODULARIDAD

- Cohesión: medida de la fuerza funcional de un módulo o clase. Se busca que el modulo tenga la cohesión más alta posible: todos sus elementos contribuyen a la ejecución de una misma tarea.
- Acoplamiento: es la medida de la interdependencia relativa entre clases o módulos. Se busca que exista el mínimo posible de acoplamiento entre módulos: los módulos se comunican solamente por medio de mensajes.

BUENA CLASE

- El nombre refleja su intención.
- Se modela un elemento específico del dominio
- Responsabilidades bien definidas
- Alta cohesión: concordancia de las responsabilidades con el propósito de la clase. Todas las responsabilidades trabajan por el mismo objetivo
- Bajo acoplamiento: número de clases con las que se relaciona reducido, responsabilidades uniformemente distribuidas entre clases y no se concentran en una sola.

DISEÑO OO

- Objetos: entidades centrales
 - Se comunican entre sí mediante el uso de mensajes
 - El conjunto de objetos que responden a los mismos mensajes se implementan mediante clases. ○ La clase describe e implementa todos los métodos que capturan el comportamiento de sus instancias ○ La implementación está totalmente oculta (encapsulada) dentro de la clase de modo que puede ser extendida y modificada sin afectar al usuario.
 - Una clase es como un módulo
 - Es posible extender y especializar una clase (herencia)
- Principios básicos
 - Modularización: módulos fáciles de manejar que comprenden las estructuras de datos y las operaciones
 - Encapsulado: se distingue entre la interfaz de un objeto (qué es lo que hace) de la implementación (cómo lo hace)
 - Tipos de datos abstractos: agrupa todos los objetos que tienen la misma interfaz y los trata como si fueran del mismo tipo
 - Herencia: reutilización
 - Mensajes: un objeto lleva a cabo sus acciones cuando recibe un mensaje concreto, codificado de una forma simple, estándar e independiente de cómo o dónde está implementado el objeto.
 - Polimorfismo: diferentes objetos responden al mismo mensaje. El sistema determina en tiempo de ejecución qué código invocar dependiendo del tipo de objeto.

BUENA CLASE

- Nombre:
 - refleja su intención
- Abstracción:
 - modela un elemento específico del dominio
- Responsabilidades bien definidas
 - Alta cohesión:
 - Concordancia de las responsabilidades con el propósito de la clase
 - Todas las responsabilidades trabajan por el mismo objetivo.
 - Bajo acoplamiento:
 - Reducir el número de clases con las que se relaciona.
 - Distribución uniforme de las responsabilidades entre clases.
 - No localizar el control o muchas responsabilidades en una sola clase.

PATRONES GRASP

Una responsabilidad es un contrato u obligación de una clase. Los métodos se implementan para cumplir con las responsabilidades. Las responsabilidades se implementan mediante métodos que actúan solos o colaboran con otros métodos y objetos.

Un patrón es una pareja Problema/Solución con un nombre, que es aplicable a otros contextos, con una sugerencia sobre la manera de usarlo en situaciones nuevas. GRASP es un acrónimo de "General Responsibility Assignment Software Patterns". Los patrones GRASP describen los principios fundamentales de diseño de objetos para la asignación de responsabilidades.

PATRÓN EXPERTO

- Problema: ¿cuál es el principio general en la asignación de responsabilidades a objetos?
- Solución: asignar una responsabilidad al experto en información. La responsabilidad de realizar una labor es de la clase que tiene o puede tener los datos involucrados (atributos).

Ejemplo: En una aplicación de Punto de Venta, alguna clase necesita conocer el total general de la venta → ¿Quién es el responsable de conocer el total general de la venta?



Para calcular el subtotal de una línea de VentasLineadeProducto se necesitan la cantidad (VentasLineadeProducto) y el precio (Especificación de Producto). VentasLineadeProducto conoce su cantidad y está asociada a Especificación de producto, por lo tanto, VentasLineadeProducto debería determinar el subtotal dado que es el experto en la información.

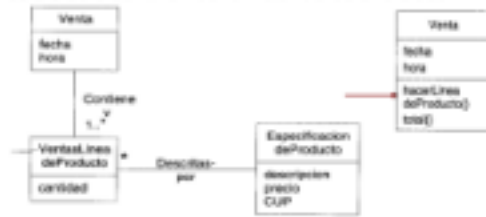
VENTAJAS

- Se mantiene el encapsulamiento de la información, ya que los objetos utilizan su propia información para realizar las tareas.
- Favorece el bajo acoplamiento, que conduce a sistemas más robustos y mantenibles
- El comportamiento se distribuye entre las clases que tienen la información necesaria información requerida, fomentando así definiciones de clase "ligeras" más cohesivas que son más fáciles de entender y mantener. - Normalmente se admite una alta cohesión.

PATRÓN CREADOR

- Problema: ¿Quién es el responsable de crear una instancia de una clase?
- Solución: se le asigna a la clase B la responsabilidad de crear una instancia de la clase A si:
 - B contiene a A
 - B es una agregación o composición de A
 - B almacena a A
 - B tienen los datos de inicialización de A (datos que requiere su constructor)
 - B usa a A

Ejemplo: En la aplicación del punto de venta, ¿quién debería encargarse de crear una instancia `VentasLineadeProducto`? Desde el punto de vista del patrón Creador, deberíamos buscar una clase que agregue, contenga y realice otras operaciones sobre este tipo de instancias. → ¿Quién debería ser el responsable de crear una nueva instancia de una clase?



Dado que una `Venta` contiene (agrega) muchos objetos `VentasLineadeProducto`, el patrón Creador sugiere que la `Venta` es un buen candidato para tener la responsabilidad de crear instancias de `VentasLineadeProducto`.

La creación de instancias es una de las actividades más comunes en un sistema orientado a objetos. Si se asignan de manera correcta, el diseño puede soportar un bajo acoplamiento, mayor claridad, encapsulamiento y reutilización.

Su principal ventaja es que se admite un bajo acoplamiento, lo que implica menores dependencias de mantenimiento y mayores oportunidades de reutilización. El acoplamiento no se incrementa dado que la clase creada ya es visible para la clase creadora (las asociaciones que motivaron su elección como creador ya eran existentes)

PATRÓN CONTROLADOR

- Problema: ¿quién debería ser responsable de manejar un evento de entrada en el sistema? (un evento de entrada es generado por un actor externo)
- Solución: asignar la responsabilidad de controlar el flujo de eventos del sistema a clases específicas, lo que facilita la centralización de actividades (validación, seguridad, etc.). El controlador no realiza estas actividades, las delega en otras clases con las que mantiene un modelo de alta cohesión. Un error muy común es asignarle demasiada responsabilidad y alto nivel de acoplamiento con el resto de los componentes del sistema.
- Opciones válidas:
 - El sistema global (controlador de fachada)
 - La empresa u organización global (controlador de fachada)
 - Algo en el mundo real que es activo y que pueda participar en la tarea (controlador de tareas)
 - Un manejador artificial de todos los eventos del sistema de un caso de uso, generalmente denominado "Manejador<nombreCasoDeUso>" (Controlador de caso de uso)

BAJO ACOPLAMIENTO

PREMISAS

- Poca dependencia entre las clases
- Si todas las clases dependen entre sí no es posible la reutilización
- Mal diseño: herencia profunda
- Se deben considerar las ventajas de la delegación respecto de la herencia

VENTAJAS

- No se ve afectado por cambios en otros componentes
- Simple de entender en forma aislada
- Conveniente para la reutilización

ALTA COHESIÓN

La cohesión es una medida de cuán fuertemente relacionadas están las responsabilidades de un elemento. Un elemento con responsabilidades muy relacionadas, y que no hace una gran cantidad de trabajo, tiene una alta cohesión. Una clase con baja cohesión hace muchas cosas no relacionadas o hace demasiado trabajo difícil de comprender, reutilizar y mantener, y está constantemente afectada por el cambio.

- Problema: ¿cómo mantener la complejidad manejable?
- Solución: asignar la responsabilidad para que la cohesión permanezca alta.

DISEÑO ARQUITECTÓNICO

La arquitectura de software de un sistema es un conjunto de estructuras necesarias para discutir sobre el sistema, que compara los elementos, sus relaciones y propiedades. Los sistemas complejos están compuestos por subsistemas que interactúan bajo el control de un diseño de sistema.

La arquitectura es una abstracción y todo sistema tiene una. Sus componentes son los subsistemas que componen el sistema, las interfaces y las reglas de interacción entre ellos. Ventajas: comunicación entre stakeholders, decisiones tempranas de diseño y reutilización.

La arquitectura es importante porque:

- Inhibe o habilita los atributos de calidad de un sistema (predicciones tempranas)
- Permite razonar y administrar los cambios a medida que el sistema evoluciona.
- Su documentación facilita la comunicación entre los stakeholders.
- Decisiones de diseño tempranas y fundamentales.
- Restricciones de la implementación.
- Base para un prototipado que evoluciona.
- Razonamiento sobre costos y cronograma.
- Modelo transferible
- Modelo reutilizable
- Ensamblado de componentes.
- Creatividad de desarrolladores: reduce la complejidad del sistema y de su diseño.
- Posibilita el entrenamiento de un nuevo miembro del equipo.

La medida en que un sistema alcanza sus requisitos de calidad depende de las decisiones de arquitectura. La arquitectura es crítica para alcanzar los atributos de calidad: las cualidades del producto deben diseñarse como parte de la arquitectura. Un cambio en la estructura que mejora un atributo de calidad suele afectar los otros atributos. La arquitectura sólo puede permitir, no garantizar, que cualquier requisito de calidad se alcance.

El estilo y estructura particular elegido para una aplicación dependen fuertemente de los requerimientos no funcionales: performance, seguridad, disponibilidad, mantenibilidad, flexibilidad, portabilidad, reutilización, legibilidad, corrección, usabilidad, eficiencia, confiabilidad, adaptabilidad, facilidad de prueba e interoperabilidad.

INTERESADOS

Los objetivos organizacionales y las propiedades del sistema requeridas por el negocio raramente se comprenden y menos aún se articulan completamente.

Los requisitos de calidad del cliente casi nunca se documentan, lo cual resulta en: objetivos que no se alcanzan y conflicto inevitable entre los interesados. Los arquitectos deben identificar e involucrar activamente a los interesados de modo de:

- comprender las restricciones reales del sistema;
- administrar las expectativas de los interesados;
- negociar las prioridades del sistema;
- tomar decisiones de compromiso.

PROCESO DE MODELADO ARQUITECTÓNICO

- Definir los requerimientos: Crear un modelo desde los requerimientos basado en los atributos de calidad esperados
- Diseño de la Arquitectura: Definir la estructura y las responsabilidades de los componentes
- Validación: Probar

la arquitectura con los requerimientos actuales y cualquier posible requerimiento a futuro.

PATRONES ARQUITECTÓNICOS

Colección de decisiones de diseño arquitectónico con nombre específico, aplicables a problemas de diseño recurrentes y parametrizadas. Se utilizan en diferentes contextos de desarrollo de software. Los patrones arquitectónicos son decisiones de diseño efectivas para organizar ciertas clases de sistemas de software, que son configurables y necesitan ser instanciadas con los componentes y conectores particulares a una aplicación.

Su propósito es compartir una solución probada, ampliamente aplicable a un problema particular de diseño. Se presenta en forma estándar y es fácilmente reutilizado. Establece las relaciones entre un contexto, un problema (atributos de calidad) y una solución (con abstracción apropiada): un conjunto de elementos o mecanismos de interacción, diseño topológico de los componentes o un conjunto de restricciones semánticas.

PATRÓN EN CAPAS

- Contexto: Sistema complejo que requiere el desarrollo de partes de manera independiente. Requieren la separación clara y bien documentada de los aspectos involucrados de manera que los módulos puedan ser desarrollados y mantenidos independientemente.
- Problema: El software requiere ser segmentado en módulos a desarrollar por separado con mínima interacción entre las partes. Soporta portabilidad, modificabilidad y reutilización
- Solución: divide el software en unidades denominadas capas. Cada capa agrupa a módulos que ofrecen un conjunto cohesivo de servicios. La relación entre las capas debe ser unidireccional (allowed-to-use).

Se definen protocolos mediante los que interactúan las capas. Modelo estricto: una capa sólo utiliza servicios de la inmediata inferior. Modelo relajado: se pueden saltar capas.

Las ventajas son que se facilita la comprensión, el mantenimiento, la reutilización y la portabilidad. La desventaja es que puede resultar complejo estructurar en capas e identificar los niveles de abstracción a partir de los requerimientos.

• Ejemplo: Web de datos

Compatibilidad descendente

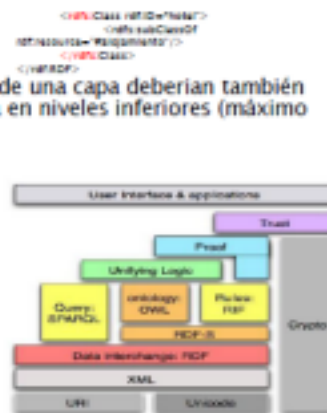
- Agentes con el conocimiento propio de una capa deberían también interpretar y usar información escrita en niveles inferiores (máximo provecho)

• Capa XML

- Base sintáctica.

• Capa RDF-Schema

- Provee primitivas de modelado para la organización de recursos en jerarquías: clases, propiedades, relaciones de subclases y subpropiedades y restricciones de dominio y rango.
- Esta basado en RDF
- Lenguaje primitivo para la definición de ontologías



PATRÓN CENTRADO EN DATOS

- Contexto: varios componentes necesitan compartir y manipular grandes cantidades de datos. Esos datos no pertenecen exclusivamente a ninguno de los componentes.
- Problema: ¿Cómo pueden los sistemas almacenar y manipular datos persistentes que son accedidos por múltiples componentes independientes?
- Solución: la interacción está dominada por el intercambio de datos persistentes entre múltiples entidades que acceden a esos datos y al menos un almacenamiento de datos compartido. El tipo de conector es lectura y escritura.

Sus ventajas son que promueve la capacidad de integración y que es posible cambiar componentes existentes y agregar nuevos componentes clientes que operan en forma independiente. Sus debilidades son que el almacenamiento de datos puede ser un cuello de botella y que productores y consumidores de datos pueden estar estrechamente acoplados.

La relación attachment determina a qué componente se conecta con qué almacenamiento de datos. Tiene como restricciones que los componentes que acceden a los datos interactúan con el almacenamiento de datos.

PATRÓN MODELO-VISTA-CONTROLADOR

- Contexto: software para IU es la porción modificada con mayor frecuencia de una aplicación interactiva. Es recomendable mantener las modificaciones de la IU separada del resto del sistema.
- Problema: ¿Cómo puede mantenerse separada la funcionalidad de IU de la funcionalidad de la aplicación y seguir siendo sensible al ingreso del usuario, o a los cambios en la aplicación de datos subyacente? ➤ Solución: El patrón MVC separa la funcionalidad de la aplicación en:

- Un modelo, que contiene los datos de la aplicación
- Una vista que muestra parte de los datos subyacentes y que interactúa con el usuario
- Un controlador, que hace de intermediario entre el modelo y la vista y administra los cambios en el estado y sus notificaciones.

La relación notifies conecta instancias del modelo, vista y controlador notificando elementos de cambios de estado relevantes. Restricción: debe haber por lo menos una instancia de modelo, una de vista y una de controlador. Debilidad: la complejidad puede no valer la pena para interfaces de usuario simples.

Ventajas:

- se presenta la misma información de distintas formas.
- Las vistas y comportamiento de una aplicación deben reflejar las manipulaciones de los datos de forma inmediata.
- Debería ser fácil cambiar la interfaz de usuario (incluso en tiempo de ejecución).
- Permitir diferentes estándares de interfaz de usuario o portarla a otros entornos no debería afectar al código de la aplicación

• Ejemplo:

- Los datos de una hoja de cálculo pueden mostrarse de en formato tabular, con un gráfico de barras, con uno de torta.
- Los datos son el modelo.
- Si cambia el modelo, las vistas deberían actualizarse en consonancia.
- El usuario manipula el modelo a través de las vistas.
(en realidad, a través de los controladores)



falta patron orientado a servicios

OTROS

Pipeline, orientada a servicios, dirigida por eventos y cliente servidor.

EVALUACIÓN DE ARQUITECTURAS

Cambiar la arquitectura de un producto ya construido requiere mucho esfuerzo, por lo que es importante evaluar la arquitectura antes de implementarla completamente, verificando los requisitos de calidad establecidos. Evaluaciones a posteriori resultan útiles como forma de aprendizaje y estudio de posibilidades de mejora, por ej. para una nueva versión del producto.

Software Engineering Institute(SEI) propone:

- Architecture Tradeoff Analysis Method (ATAM)
- Software Architecture Analysis Method(SAAM)

ARCHITECTURE TRADEOFF ANALYSIS METHOD (ATAM)

Establece en qué medida una arquitectura particular satisface las metas de calidad. Provee ideas de cómo esas metas de calidad interactúan entre ellas, como realizan concesiones.

Fase	Descripción	Participantes	Duración Típica
0 – <i>Partnership y Preparación</i>	Logística, planificación, reclutamiento de stakeholders, formación de equipos.	Líder del equipo de evaluación y tomadores de decisiones claves del proyecto	Procede informalmente por demanda, quizás a lo largo de unas pocas semanas
1 – <i>Evaluación</i>	Pasos del 1 a 6	El equipo de evaluación y los tomadores de decisiones claves del proyecto	1 o 2 días seguidos por un intervalo de 2 o 3 semanas
2 – <i>Evaluación</i>	Pasos del 7 a 8	El equipo de evaluación, los tomadores de decisiones claves del proyecto y los stakeholders	2 días
3 – <i>Seguimiento</i>	Generación y entrega de reporte, y mejora de procesos	El equipo de evaluación y el cliente de la evaluación	1 semana

1. Presentar ATAM
 2. Presentar los drivers del negocio
 3. Presentar la arquitectura
 4. Identificar los enfoques arquitectónicos
 5. Generar el árbol de utilidad de los atributos de calidad
 6. Analizar los enfoques arquitectónicos
 7. Brainstorming y priorización de escenarios
 8. Analizar los enfoques arquitectónicos
 9. Presentar Resultados
- Fase 1: 1, 2, 3
 Fase 2: 4, 5, 6, 7, 8
 Fase 3: 9

VERIFICACIÓN

Proceso de comprobación para establecer la confianza en que el sistema de software es adecuado

- comienza ni bien están disponibles los requerimientos.
- continúa a través de todas las etapas del proceso de desarrollo.

INSPECCIONES DE SOFTWARE:

TECNICAS ESTATICAS:

- No requieren que el sistema se ejecute.
- Analizan y comprueban:
 - Representaciones del sistema: ERS, diagramas de diseño, código fuente del programa.
- Pueden aplicarse en todas las etapas del proceso.
- Para descubrir errores, se utiliza:
 - conocimiento del sistema
 - dominio de aplicación
 - lenguaje de modelado/aplicación

TECNICAS DINAMICAS:

- Requieren un prototipo ejecutable del sistema.
- Implican ejecutar una implementación del software con datos de prueba.
- Se examinan:
 - Salidas
 - Entorno

PLANIFICACIÓN

Inicia en las primeras etapas del ciclo de desarrollo.

- Define checklists para las inspecciones
- Define el plan de pruebas (PP).

El plan de pruebas contiene:

- Descripción del proceso de pruebas.

- Fases de pruebas a realizar.
- Requerimientos a probar.
 - Ej. casos de uso.
- Productos de software a probar.
- Calendarización de las pruebas.
- Procedimientos de registro de las pruebas:
 - Registro de defectos, asignación, tiempos estimados de corrección, etc.
- Requerimientos de hardware y software.
- Restricciones que afectan al proceso de pruebas
 - Ej personal dedicado.

PROCESO GENERAL DE PRUEBAS:

Se realiza en cuatro etapas:

1. Pruebas de las unidades (pruebas de métodos y clases)
se prueban programas individuales como funciones u objetos.
2. Pruebas de integración o de subsistemas:
se prueban agrupaciones de clases relacionadas.
3. Prueba de sistema:
se prueba el sistema como un todo.
4. Prueba de aceptación:
prueba del sistema en el entorno real de trabajo con intervención del usuario final.

PRUEBAS CAJA NEGRA O FUNCIONALES:

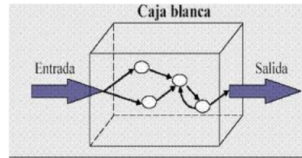
Pruebas de caja negra:

- Es una política de selección de casos de pruebas basado en la especificación del componente o programa.
- Las pruebas se seleccionan en función de la especificación y no de la estructura interna del software.



PRUEBAS CAJA BLANCA O ESTRUCTURALES:

- Se seleccionan en función del conocimiento que se tiene de la implementación del módulo.
- Se suelen aplicar a módulos pequeños.
 - El tester analiza el código y deduce cuántos y qué conjuntos de valores de entrada han de probarse para que al menos se ejecute una vez cada sentencia del código.
- Se pueden refinar los casos de prueba que se identifican con pruebas de caja negra.



PROCESO DE DEPURACIÓN:

- Proceso que localiza y corrige los errores descubiertos durante la verificación y validación.
- Proceso independiente.
- No siempre los errores se detectan cerca del punto en que se generaron.
- Las herramientas de depuración facilitan el proceso.
- Después de reparar el error, hay que volver a probar el sistema (pruebas de regresión).
- La solución del primer fallo puede dar lugar a nuevos fallos.

PRUEBAS DE SISTEMA: implican integrar varios componentes y probar el sistema integrado. Existen dos fases de pruebas de sistema:

- Pruebas de Integración:
 - Equipo de testing tiene acceso al código del sistema.
 - Cuando descubre un problema, se intenta localizar el componente que lo origina y que debe ser depurado.
 - Objetivo: encontrar defectos en el sistema.
- Pruebas de Entrega:
 - se prueba la versión a entregar al cliente sin disponer del código.
 - Son pruebas de caja negra
 - Cuando participa el cliente en estas pruebas, se denominan Pruebas de Aceptación.
 - Si la entrega es lo suficientemente buena, el cliente la acepta para su uso.

MONKEY TESTING:

Técnica de testing de caja negra

- ☐ Ej. campos de un formulario
- ☐ Se les proporciona datos de entrada aleatorio

TIPOS:

- Dumb Monkey Testing
- Brilliant Monkey Testing
- Smart Monkey Testing.

ESCENARIOS DE CASOS DE USO:

- Son instancias de un caso de uso.
- Describen un camino completo del caso de uso
 - con alternativas y excepciones
 - sin alternativas y excepciones

- Debe:

- Generar un escenario por combinación de flujos básicos y alternativos del caso de uso.
- Determinar lo que se espera que el sistema haga, y para quién y con quién lo hará.
- Especificar escenarios de uso que cubran todos los posibles caminos a través de las funciones del sistema.

GENERACIÓN DE CASOS DE PRUEBA:

- **Caso de prueba:**

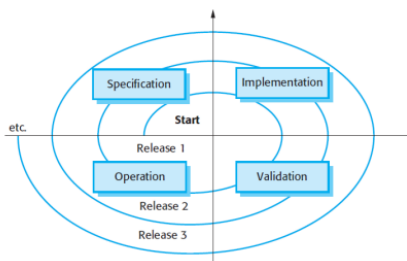
- conjunto de entradas, condiciones de ejecución y resultados esperados desarrollados para un objetivo particular
- ejercitar o verificar la conformidad de una funcionalidad específica.
- Son necesarios para verificar la implementación exitosa y aceptable de los requerimientos del producto.

- **Proceso de 3 pasos:**

1. Para cada caso de uso, generar un conjunto completo de escenarios de caso de uso.
2. Para cada escenario identificar al menos un caso de prueba y las condiciones que lo harían "ejecutarse".
3. Para cada caso de prueba, identificar los valores de los datos con los cuales probar la funcionalidad.

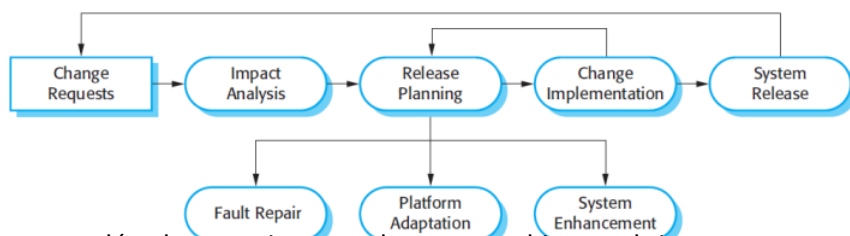
EVOLUCIÓN

Se puede pensar en la ingeniería del software como un proceso en espiral con actividades que se realizan continuamente durante el tiempo de vida del sistema.



Los procesos de evolución incluyen las siguientes actividades fundamentales:

- Análisis de cambios
- Planificación de entregas
- Implementación de los cambios
- Entrega del sistema a los clientes
- Análisis del impacto: En este proceso se evalúa el costo e impacto de estos cambios en el sistema.
- Durante la planificación de entregas, se consideran todos los cambios propuestos (reparación de defectos, adaptación y nuevas funcionalidades).
- A continuación, se decide en qué lenguajes se va a implementar la siguiente versión del sistema.
- Se implementan y validan los cambios. y se entrega una nueva versión del sistema



MANTENIMIENTO:

El mantenimiento del software es el proceso general de cambiar un sistema después de que éste ha sido entregado.

- Administrar el proceso de cambios.

- Tipos:

- Reparar fallas
- Adaptación ambiental
- Adición de funcionalidad

- **Reparar fallas.**

- Modificación de un producto software después de su puesta en producción (entrega) para corregir los fallos que se han descubierto.
 - Los errores de código son baratos para reparar.
 - Los errores de diseño son más caros dado que puede implicar la reescritura de varios componentes.
 - Los errores de requerimientos son los más caros de reparar debido a que pueden implicar un rediseño extensivo.

- **Adaptación ambiental.**

- Mantener operativo un software cuando se realizan cambios en el entorno de producción.
- Se requiere cuando algún aspecto del entorno del sistema, como el hardware, la plataforma operativa del sistema u otro soporte, cambia el software.
- El sistema de aplicación tiene que modificarse para lidiar con esos cambios ambientales.
- Cambiar el entorno en el que se utiliza el software (que incluye el sistema operativo, la plataforma de hardware, navegador), para mantener su plena funcionalidad en estas nuevas condiciones.

- **Adición de funcionalidad.**

- Es necesario cuando varían los requerimientos del sistema, en respuesta a un cambio organizacional o empresarial.

MANTENIMIENTO. COSTOS

